

A work product pool approach to methodology specification and enactment

Cesar Gonzalez-Perez ^{*}, Brian Henderson-Sellers

Department of Software Engineering, Faculty of Information Technology, University of Technology, Sydney, P.O. Box 123, Broadway NSW 2007, Australia

Received 27 March 2007; received in revised form 25 September 2007; accepted 6 October 2007

Available online 13 October 2007

Abstract

Software development methodologies advocated and used today, whether traditional and plan-based or contemporary and agile, usually focus on process steps i.e. they start with requirements and iteratively describe what steps are necessary to move to the next stage or phase, until the software application is delivered to the end user. Such a process-oriented view of methodologies, based on the metaphor that human organizations are “machines” that “execute” processes, often results in methodologies that are too rigid and hard to follow, and most often than not end up being ignored or bypassed. Our proposal here is that, since the ultimate aim of software development is to provide a software *product*, software development methodologies should be described in terms of the intermediate products that are necessary to reach such a final product, plus the needed micro-processes that, as necessary evils, will be required to produce the appropriate work products from other, previously created ones. Using this product-oriented approach, software development methodologies can be specified that are, at least, as flexible as lightweight, agile approaches and, at the same time, as powerful and scalable as plan-oriented ones.

© 2007 Elsevier Inc. All rights reserved.

Keywords: Software development methodologies; Enactment; Metamodelling; ISO/IEC 24744

1. Introduction

Those who dedicate their professional life to the study and construction of software development methods¹ are condemned to repeatedly find themselves in the awkward

situation of having to tell other people how to do their jobs. In fact, specifying a methodology is, simply, stating how somebody should do their job. A methodology specifies the products to be developed and used, the steps to follow in changing existing products and creating new ones, the different roles that will perform these tasks, and perhaps the quality mechanisms to utilize. It is not surprising that software developers, the ultimate users and beneficiaries of methodologies, do not like having a methodology imposed on them, because this means that somebody is telling them how they should be doing their work. Users of methodologies often complain that methodologies are too prescriptive and rigid (Nandhakumar and Avison, 1999; Abrahamsson et al., 2002; Avison and Fitzgerald, 2003; Chau et al., 2003), that they are not realistic, that they make them do things that are a waste of time, focus on “ceremony” rather than software creation and that the work style fostered by the methodology does not match the organizational culture of their workplace.

^{*} Corresponding author. Address: Rua Buxo, U-11-B, bajo C, Os Tilos, Teo 15894, Spain. Tel.: +34 677 024545.

E-mail addresses: cesargon@verdewek.com (C. Gonzalez-Perez), brian@it.uts.edu.au (B. Henderson-Sellers).

¹ A note on terminology. Different authors use varying (and often loose) terminology. Here, method (a.k.a. methodology) is a description of *all* elements (aspects of process, produce, people and tools) needed for software development. It is also often called a process model, specified by a metamodel – e.g. (Dowson, 1987; ISO/IEC, 2007). Enacting this process model or methodology on a real endeavour (e.g. a project) is called here a “process”. This is also in agreement with the notion of process and capability assessment, which also uses processes in terms of their enactment. However, sometimes, we use “process” as shorthand for “process model”.

Agile methodologies have emerged as a response to these issues. Most software developers today are actually software *programmers* and, logically, they gravitate towards writing source code rather than spending time in drawing diagrams or thinking about requirements. Agile methods, usually focussed on providing something that works in the shortest possible time, fit well into this tendency, focussing on a rapid response to changing requirements (Dove, 1997; Wong and Whitman, 1999; Newkirk, 2002; Bettin, 2003), working code, people and customer collaboration and involvement. Also, agile methods promise a large degree of freedom from the apparent prescriptiveness, rigidity and naivety of plan-oriented approaches. Most success stories tend to be anecdotal (Turk et al., 2005); although there are a few research-focussed empirical studies (Lindvall et al., 2002; Salo and Abrahamsson, 2004; Henderson-Sellers and Serour, 2005) that show that agile methods work well in certain settings, especially where small, co-located teams of very smart developers work in close collaboration with a willing customer (Boehm, 2002; Highsmith and Cockburn, 2001).

Agile methods, however, are also perceived to have shortcomings. Their lack of scalability and their inability to produce complex and robust architectures have been clearly documented; e.g. (Constantine, 2002; Reifer et al., 2003; Auvinen et al., 2006). Also, the lack of documentation that usually accompanies agile approaches is sometimes feared by management and sometimes found to be a real problem when the original team is disbanded and the product needs to be maintained (McBreen, 2002). Some authors have proposed that an optimum degree of agility can be determined for any specific organizational and technical situation (Henderson-Sellers et al., 2007), so that an optimal methodology that strikes a perfect balance between agility and ceremony can be, at least theoretically, devised (Gonzalez-Perez, 2003). According to this view, there seems to be a continuum that extends from the most agile approaches that can ever be to, at the other extreme, the heaviest, most ceremonial methodologies. Finding the appropriate trade-offs and determining the “sweet spot” along this continuum for each individual case is the job of the methodologist.

Whether or not the above perceptions regarding the “flaws” in traditional, plan-based methods and agile approaches are valid, it would be nice to be able to combine the nimbleness and responsiveness of agile methods with the scalability and power of plan-oriented ones. In this paper we present a new approach to the specification and application of methodologies that, according to our experience, provides such an advantage. In order to contextualize our work and set the scene for further explanation, Section 2 takes a step back to look at the present and past of software development methodologies with fresh eyes, describing in detail the major issues that we find with them. Section 3 introduces our solution to these issues, called the work product pool approach, and relates it to previous work by other authors. Section 4 describes the algorithms and techniques used by the work product pool approach

to achieve its goals, and Section 5 explains how a software tool was implemented to support this approach. Finally, Section 6 provides the conclusions.

2. Methodologies revisited

This section takes a step back and looks at software development methodologies, taking into account how they have evolved over the last couple of decades. As noted above, it is customary nowadays to differentiate between agile methodologies and conventional, plan-oriented ones. Or, at least, between degrees of agility (or ceremony, depending on the perspective) (Qumer and Henderson-Sellers, *in press*). Agile approaches have arisen because of the perceived excess of prescriptiveness and rigidity in conventional methodologies; however, some authors have argued that agile approaches are as prescriptive as plan-oriented ones, only that they prescribe different things (Boehm, 2002). This being the case, we can argue that the perceived freedom that agile methods provide does not come from their lack of prescriptiveness but from the fact that they prescribe what the developers would do anyway.

According to this line of argument, methodologies (agile and non-agile) are usually too prescriptive and rigid. This is especially true of the process-oriented, time-ordered aspects in them, such as the sequences of steps to be performed. The analogy with a computer program is often applied: a conventional methodology can be described as a sequence of steps to be executed. In fact, several software engineering communities use the term *process* to refer to methodologies, revealing an underlying, although not always explicit, metaphor that sees humans as machines that execute methodologies in an almost mechanistic manner. Processes specified for machines, such as those often found in chemical engineering and related fields, make extensive use of concepts such as sequences, branches, loops and communication between parallel processes; some argue that a methodology can also be expressed following this approach. Gantt-inspired delivery processes in RUP – such as (Kruchten, 2003) – or the work breakdown-based concepts in SPEM² 2 (OMG, 2007) are some notable attempts. Even the switch to “practices” advocated by (Jacobson et al., 2007) still retains a process (methodology) focus. The big difference is that methodologies are carried out by humans, who are “soft” and whimsical, rather than machines. In addition, process-focussed methodologies often describe only the “normal” case, i.e. the flow of events that is expected to happen if everything works fine. However, experience shows us that life is messy and complicated, and that unanticipated circumstances are more frequent than desirable. The variability space in a machine is large and hard to cope with. Thus, anticipating all the exceptional situations in a machine-executable process is difficult; the variability space in a society of individuals

² SPEM = Software Process Engineering Metamodel.

is, at least, larger than the former, and, for all practical purposes, unfathomable.

There is a large community advocating software process improvement, centred around standards such as SPICE³ (ISO/IEC, 2004) and CMM⁴ (Software Engineering Institute, 2002), that believes that the quality of software products can be improved by improving the quality of the process that people follow in their development, as advocated by (Humphrey, 1989). Unfortunately, as (Greenfield and Short, 2004) point out, the capabilities of existing software platforms are usually beyond the capabilities of the tools we have. According to this, we argue that software engineers know, by analytical reasoning rather than by experience, that existing platforms can support and execute systems of much higher power and quality than those we can build today, causing endless frustration. In other words, we are systematically underexploiting our infrastructures. To the process improvement believer, this is an irresistible problem, since the hypothesis says that better products can be obtained by perfecting the process. This has led to the specification of highly complex processes, which, rather than rendering better products, often increase the divide between people and process.

In addition, software process improvement heavily relies on the idea that measuring the process makes sense. The argument goes like this: since the quality of the process determines (at least, to a large extent) the quality of the product, measuring the process can give us some information about the product. We do not necessarily disagree with this, but we also think that the product itself must be measured. If an organization systematically develops products of high quality, on time and under budget, the process being followed, although importantly ever-present, becomes “invisible”. In this respect, the products are more tangible, visible and communicable than processes. Products are usually expressed in some persistent medium (such as paper or computer files), can be processed by machine with relative ease, and sent by email or distributed through the Internet. Processes cannot. Processes are measured through evidence that they have been performed; this evidence being, usually, products. This is the rationale underpinning capability assessments e.g. SPICE with its focus on measurable *outcomes*. It is a contradiction that we are measuring processes to obtain information about the products, and that the only way to measure processes is via products. Why not measure products directly, if we are counting on them anyway?

At this point, we need to realize that the ultimate objective of software development is the systematic production of reasonable, acceptable products. Therefore, the goal of any software development methodology should be obtaining a product. This is in glaring contrast with most current methodologies, which seem to focus on specifying how to

perform a process rather than how to obtain a final product. If the desired product could be magically obtained without any effort, without performing any work, we would do it. Process is not needed *per se*, but is a necessary evil that we commonly perform as a way to obtain what we really want: a product. In summary, methodologies should give greater emphasis to obtaining products, not performing processes.

Of course, an emphasis on product rather than process is not entirely novel. For example, (Shere and Carlson, 1994) recommend viewing phases “in terms of the input and output data associated with each” – although the ASSET methodology they build upon this statement is still described using process language, the seven steps of ASSET commencing (1) Perform ..., (2) Perform ..., (3) Combine ..., (4) Define ..., (5) Define ..., (6) Define ... and (7) Balance. Humphrey and Kellner (1989) propose an entity process model (EPM) where entities are “things that persist rather than ephemeral objects that are transiently introduced within the process”. Yet, despite such examples, a process way of describing and thinking about software development has dominated both research and practice for the last several decades (consider, for example, the proliferation of software engineering conferences with “process” in their title).

A consequence of the advantages described in the previous paragraphs is that a methodology that focuses on expressing its products rather than its process is arguably more people-oriented, being better at dynamically reorganizing the work to be done and opportunistically exploiting unforeseen circumstances. In other words, this is a highly *agile* methodology.

As a conclusion to this section, it is our hypothesis that methodologies should be expressed in terms of the products they deal with, rather than the processes that are to be performed; and that, by doing so, methodologies will be as nimble and adaptive as agile approaches but as robust and scalable as plan-oriented methodologies. The following section describes a specific approach that implements this philosophy.

3. The work product pool approach

A methodology can be expressed in terms of the products to be obtained rather than the process to be performed by adopting a backtracking approach by which, starting from the final deliverable to be obtained, the question “what products are necessary to easily and directly obtain this product?” is repeatedly asked and answered. Let us consider, for the sake of simplicity, a highly generic, utterly simple methodology, named “SimpleMethod”, owned and used by an organization named “SimpleOrg”. The aim of SimpleMethod is to develop software systems. Each time SimpleOrg applies the methodology, the ultimate objective they pursue is a finished software system (Fig. 1).

Now, the question must be asked: what products are necessary to easily and directly obtain the finished software

³ SPICE = Software Process Improvement and Capability dEtermination.

⁴ CMM = Capability Maturity Model.

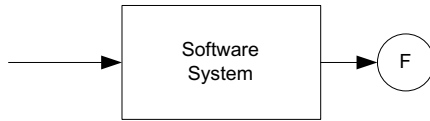


Fig. 1. The ultimate objective of SimpleMethod is a finished software system. The box represents a product. The arrow represents the notion of purpose: there is the purpose of obtaining the software system. The letter “F” in a small circle means final fulfilment, i.e. the purpose of the source product (the software system in the example) is to fulfil the final objective of the methodology.

system? Depending on the skills, business model, organizational style and other parameters of SimpleOrg, different approaches may be possible. Let us imagine that the technology employed and the skills of SimpleOrg are such that a finished software system can be easily and directly produced from a class model and a use case specification (Fig. 2).

The question must be asked again for each of the two newly introduced products: what products are necessary to easily and directly obtain the class model? Let us assume that a requirements specification is enough for this. Similarly, what products are necessary to easily and directly obtain the use case specification? The response is that the requirements specification is needed. The reusable asset pool of SimpleOrg is also necessary in order to reuse common use cases that recurrently appear in the SimpleOrg’s target market. In turn, the same question must be asked of the requirements specification and the reusable asset pool; in the first case, we find that the requirement speci-

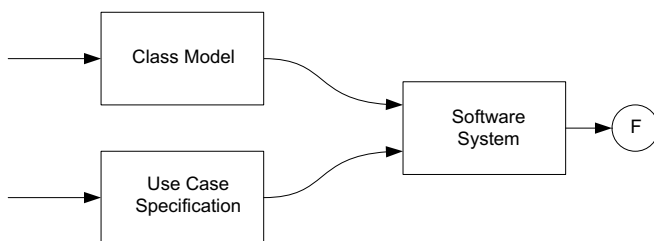


Fig. 2. The finished software system can be obtained from a class model plus a use case specification. The arrows, which represent the notion of purpose, indicate that the purpose of these two products is the production of the final system.

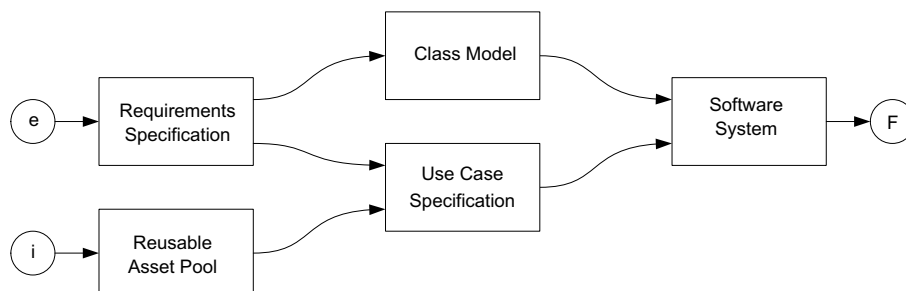


Fig. 3. The complete SimpleMethod methodology. The letter “e” in a small circle means that the target product is externally provided. The letter “i” in a small circle means that the target product is internally available.

cation is externally provided to SimpleOrg, so no additional products are necessary to obtain it. The reusable asset pool, similarly, is internally available in the organization, so no additional products are necessary. Since no additional products have been introduced and all the existing products can be either obtained easily and directly from other products, are externally provided, or are internally and readily available, we can say that closure has been achieved and that the methodology description is formally complete (Fig. 3).

Several remarks must be made at this point. First of all, it is evident that SimpleMethod is not a realistic methodology; a true methodology would involve many more products and a more complex network of purpose dependencies. However, the basic principles would stay unchanged: from a collection of initial products that are either internally available or externally provided, intermediate products are obtained with the purpose of obtaining some additional ones, and this is repeated until the final product is achieved. Secondly, multiple final products can exist. For example, a methodology could include a post-mortem report as an additional final product.

Thirdly, it must be noted that the structure of the methodology emerges by backtracking from the final product, which, by definition, is necessary. Since the only relationships between products are those of purpose, this approach guarantees that all the intermediate products that are generated are intrinsically necessary, and tracing between products is straightforward. For example, if the organization that produces post-mortem reports wanted to cease creating them, it would be easy to look up in the purpose dependency graph which products are purpose-wise related to this and, where possible, remove them.

Furthermore, we must point out that this approach is strongly declarative, as opposed to the conventional process-oriented style of methodology description, which is imperative. Rather than telling the developers what actions must be done, we tell them what intermediary objectives there are, and what paths can be followed to easily and directly attain each one from the existing ones. By doing so, the methodology description focuses on what is essential, namely, obtaining the final product, rather than on accidental needs, such as the specific jobs to be performed.

This does not mean that process is unimportant or non-existent; process is still needed to bridge the gaps established by purpose dependencies. For example, some kind of process will be necessary to obtain the class model from the requirements specification in Fig. 3. However, by focussing on products and purpose dependencies, this approach provides an extremely high degree of flexibility to accommodate changes to these inter-product processes; for example, imagine that the class model is typically obtained from the requirements specification by underlining the nouns used in the requirements, determining candidate classes and then playing the CRC⁵ card game until a solid class structure stabilizes. If, during a specific enactment of the methodology, the requirements are seen to be very similar to those of a past system, the class model of this past system can be reused and partially copied into the one being developed without changing the specific goal (here of creating a class model from a requirements specification) and hence the overall methodology whatsoever. Since the process at such a micro-scale is not part of the structure of the methodology, it can be freely changed as long as the overall product network stays constant. Selecting an inter-product process element (e.g. a task or a technique) allows the developer to identify a balance between a plan-based or agile approach, to select elements suitable to their skill level, to choose elements based on their maturity. In other words, there may be a recommended selection of process-focussed fragments available, answering the question “What task or technique can I utilize here?” With the addition of a knowledge base capturing both technical and personal elements of software development, it is possible, with software tool assistance, to create deontic values – e.g. (Graham et al., 1997) – for all possible process/product pairs, product/producer pairs etc. and thus receive advice not just on all the options, but also a recommendation, i.e. “What task or technique *should* I utilize here?” – e.g. (Nguyen, 2002; Henderson-Sellers and Nguyen, 2004). Other operations research techniques such as PERT (Program Evaluation and Review Technique) may also be useful – a topic for future research.

Before continuing to describe the details of the work product pool approach, we need to redefine the concept of methodology in the light of the concepts so far introduced. A methodology, at a very high level of abstraction, can thus be thought of as a collection of product specifications interrelated by purpose dependencies.

3.1. Theoretical infrastructure

If we want to go beyond this high level of abstraction and add some detail to the methodology description, some theoretical infrastructure must be put in place. Specifically, some formal support is necessary to go beyond *describing* a methodology to *specifying* it.

Until now, the only concept that we have introduced is that of product, although no definition has been provided. According to the ISO/IEC 24744 standard metamodel for methodologies (ISO/IEC, 2007), a work product is *an artefact of interest to the endeavour*, that is, something (such as a document, a physical thing or a piece of information) that is relevant for the project being carried out. Notice that ISO/IEC 24744 used the term “work product” rather than simply “product”; we will adopt the standard “work product” instead of “product” from this point on, and we equate the concept of “work product” to the definition given by the above mentioned standard for the class *Work-Product*. Furthermore, we need to distinguish between work products and work product kinds. A work product is a real product such as a physical document that I can place on my desk or a piece of software that I can write and run. A work product kind, on the other hand, is the abstraction of any work product of a given kind. For example, “Class Model” is not a work product but a work product kind, since it represents any possibly conceivable class model. Although most metamodels are quite loose with this distinction, ISO/IEC 24744 is very strict in these terms, because their correct usage is crucial to guarantee the appropriate representation of both the methodology and endeavour domains (Henderson-Sellers and Gonzalez-Perez, 2005b). In fact, the boxes in Fig. 3 (and previous figures) represent work product *kinds*, not specific work products. We will use the strictly precise terminology from this point on.

The adoption of ISO/IEC 24744 reflects the real-world domains shown in Fig. 4: the endeavour domain in which people undertake projects, create software product lines etc.; the method domain, which provides advice and tools to help people undertake endeavours; and the metamodel domain in which concepts are defined and standardized as in ISO/IEC 24744 using a powertype-based metamodel (see also Gonzalez-Perez and Henderson-Sellers, 2006c). ISO/IEC 24744 also strongly suggests taking a *method engineering* approach to methodology specification. Method engineering (Brinkkemper, 1996; Kumar and Welke, 1992) is an approach in which methodologies are not built from scratch but assembled from pre-existing method components available from a repository. This way, a repository stores well-tested and reusable method components of various types, which can be leveraged by a method engineer to incrementally compose a methodology by selecting the best fragments according to the needs and preferences of the future users of the methodology. Although the existing method engineering literature describes in detail the basic structure and concepts of method engineering (Henderson-Sellers, 2003; Ralyté et al., 2003; Ralyté and Rolland, 2001; Saeki, 2003), it systematically fails to describe the driving forces behind the method component assembly process: What is the overall motivation to select one method component rather than another? What are the steps to take in order to build a complete methodology from individual fragments? In this

⁵ CRC = class, responsibility, collaborator.

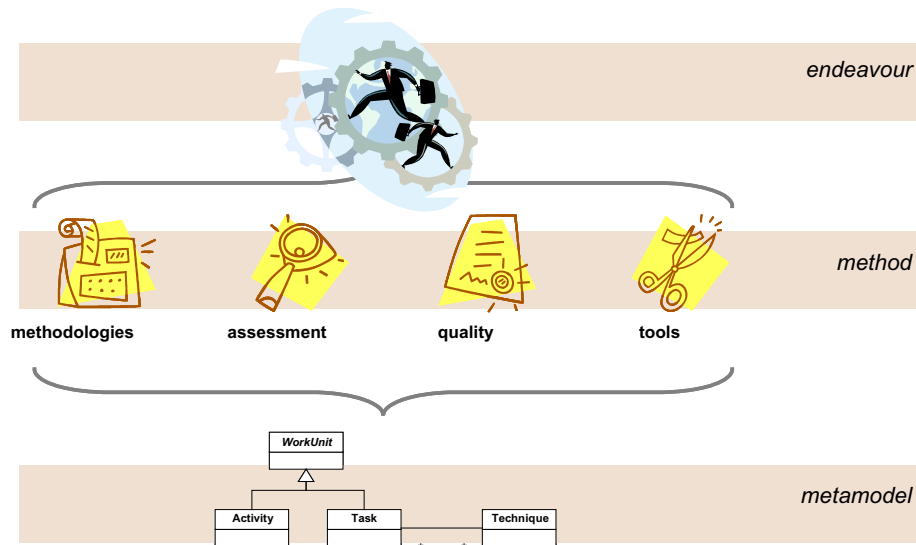


Fig. 4. The three layer architecture used to support methodologies in the context of a powertype-based metamodeling framework such as that specific in ISO/IEC 24744 (ISO/IEC, 2007). The middle layer is interchangeably referred to as “method” or “methodology” (after Henderson-Sellers, 2006).

paper, we propose that the overall motivation is satisfying purpose dependencies between work products, and the steps to take in order to attain this are described in full detail in Section 4.

3.2. The work product pool

The practical meaning of Fig. 3 is that the Simple-Method methodology involves work products of five kinds: requirements specifications, reusable asset pools, class models, use case specifications and software systems. In other words, whenever this methodology is enacted (i.e. applied to a particular endeavour: Fig. 4), work products of all of these kinds will appear. In our example, and according to the diagram in Fig. 3, a requirements specification will be externally provided, a reusable asset pool will be internally available, class models and use case specifications will be created as intermediary work products, and a software system will be created as a final work product. For some kinds of work products, multiple instances will typically appear; for example, multiple class models for the same system can exist, perhaps corresponding to different abstraction levels. Similarly, multiple use case specifications may appear, one for each individual use case. For other kinds of work products, however, only one instance will ever appear; for example, a single requirements specification is usually found within a given endeavour. Whether one or multiple work products of each kind actually appear at the endeavour level is not relevant here. In any case, the collection of existing work products at any particular point in time during the endeavour is called the work product pool.

We must make an important point here. Methodology enactment, a term used in standards documents from ISO and the OMG, entails the creation of endeavour-level ele-

ments, such as physical documents and concrete tasks. Creating a project plan, however, is not enacting a methodology. This point is important because some methodology support tools (especially those in the CMM community) use the word “enactment” to mean the creation of a project plan from the specification of a set of processes.

Returning to our main discussion, and as we have said, the boxes in Fig. 3 represent *kinds* of work products, and, for each kind, one or more work products can actually appear in the pool. From a modelling perspective, we can say that the work product kinds are the types of which the work products are instances; in addition, these types are subtypes of the more generic *WorkProduct* (Fig. 5).

From the point of view of the work being done by the software developers, we can say that the work product pool begins its existence, at the very beginning of the endeavour, containing only those work products that are externally provided or internally available. We can call these *initial* work products. From that point on, new work products can appear in the pool, and existing work products can be modified, used without modification (queried or consulted) or even deleted. The end of the endeavour is marked by the appearance of all the final work products at the appropriate level of completeness and quality. At this point, perhaps some intermediate work products remain in the pool; some are of the kind usually called “documentation”, and some others can be safely discarded.

Creating, using, modifying and deleting work products in the pool is achieved through *actions*. In ISO/IEC 24744, an action represents a usage event that a task performs upon a work product. A task, in turn, is a simple unit of work. There exist in 24,744 four different types of usages that actions can perform, which correspond to the four scenarios that we have previously identified: creation, read-only use, modification and deletion. Therefore, an

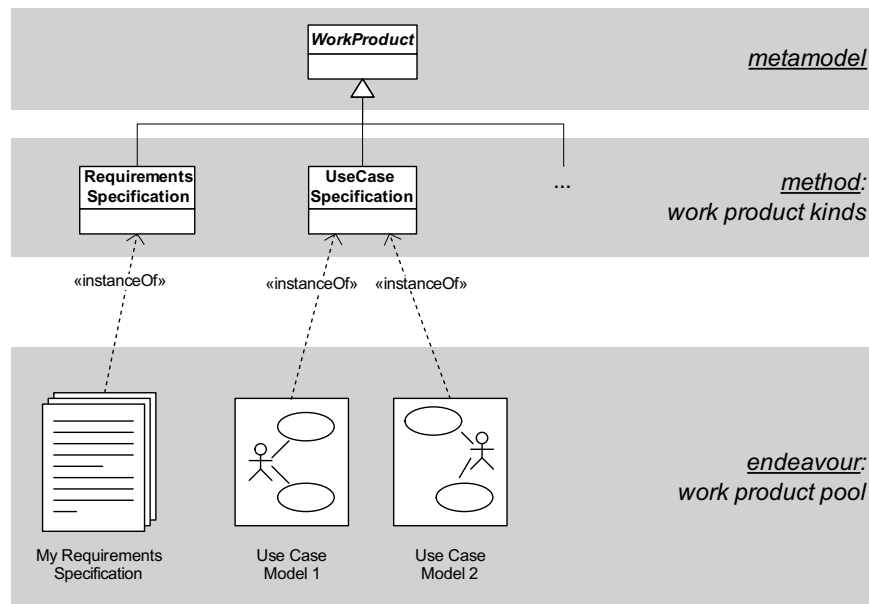


Fig. 5. Modelling work products and work product kinds. All the class diagrams in this paper use UML 2 (OMG, 2006) notation.

action can be described as a triplet formed by a task, a work product, and a usage type. For example, the action `<“Create class model”, “Class model”, Create>` means that the task “Create class model” creates a “Class model” work product. Similarly, the action `>“Create class model”, “Requirements specification”, ReadOnly` means that the same task, “Create class model”, reads the “Requirements specification” work product but does not change it.

Actions occur at the endeavour level; they are the mechanism by which the work product pool is altered, creating, changing and destroying work products. Actions, like work products, are not of random types, but can be formally described as part of the methodology specification. In the same way that work product kinds are the classes from which actual work products are instantiated, action kinds are the classes from which actual actions are instantiated (Fig. 6).

Each action acts as a “connector” between a specific task and a specific work product, performing a usage of a particular type. In our example, actions of the Create and ReadOnly usage types are illustrated. Notice that the usage type of an action is a definitional property, since two actions operating on the same task and work product but with different usage types are, in fact, very different actions! For this reason, the usage type of each particular kind of action must be fixed at the methodology level. However, using an attribute in the *Action* class would be insufficient, since that would allow each individual instance of *Action* (i.e. each individual action) to have different usage types, even when they were of the same action kind. Looking again at Fig. 6, we can see that every action that is an instance of *Action1*, for example, must have a ReadOnly usage, whereas every action that is of the *Action2* kind must have a Create usage. The usage type, therefore, is a

property of the action kind rather than the action: each action kind has a particular usage type. To the class modelling mind, the previous sentences immediately bring the idea of an *ActionKind* class with a *Type* attribute; in fact, this is what ISO/IEC 24744 specifies.

We have said that action kinds are the subclasses of *Action*; now we say that they are the instances of a new class, *ActionKind*. How can this be possible? The 24744 standard acknowledges the fact that, in a methodology, most elements are better represented by using a class and an object *at the same time*. In other words, an action kind is better represented as a class as long as the methodology user is concerned, since she will need to instantiate it into actual action objects in the endeavour; on the other hand, an action kind is better represented as an object as long as the method engineer is concerned, since this allows him to properly describe it through attribute values (Fig. 7). The class and the object, despite of being different model elements, represent *the same thing*, and they form a conceptual unity called a clabject (Atkinson, 1998; Atkinson and Kühne, 2000).

Note that the clabject-based approach can be applied to most methodology elements; in fact, work products and tasks are also modelled by ISO/IEC 24744 as pairs of classes that, at the methodology level, are “instantiated” as clabjects. A complete discussion of this approach is beyond the scope of this paper, but can be found in (Gonzalez-Perez and Henderson-Sellers, 2006c; Henderson-Sellers and Gonzalez-Perez, 2005b). An important point is worth making, however. The relationship between *Action* and *ActionKind* in Fig. 7, depicted as a simple line, is not a regular association, but represents a *power type pattern* relationship (Gonzalez-Perez and Henderson-Sellers, 2006c; Henderson-Sellers and Gonzalez-Perez, 2005a). Intuitively,

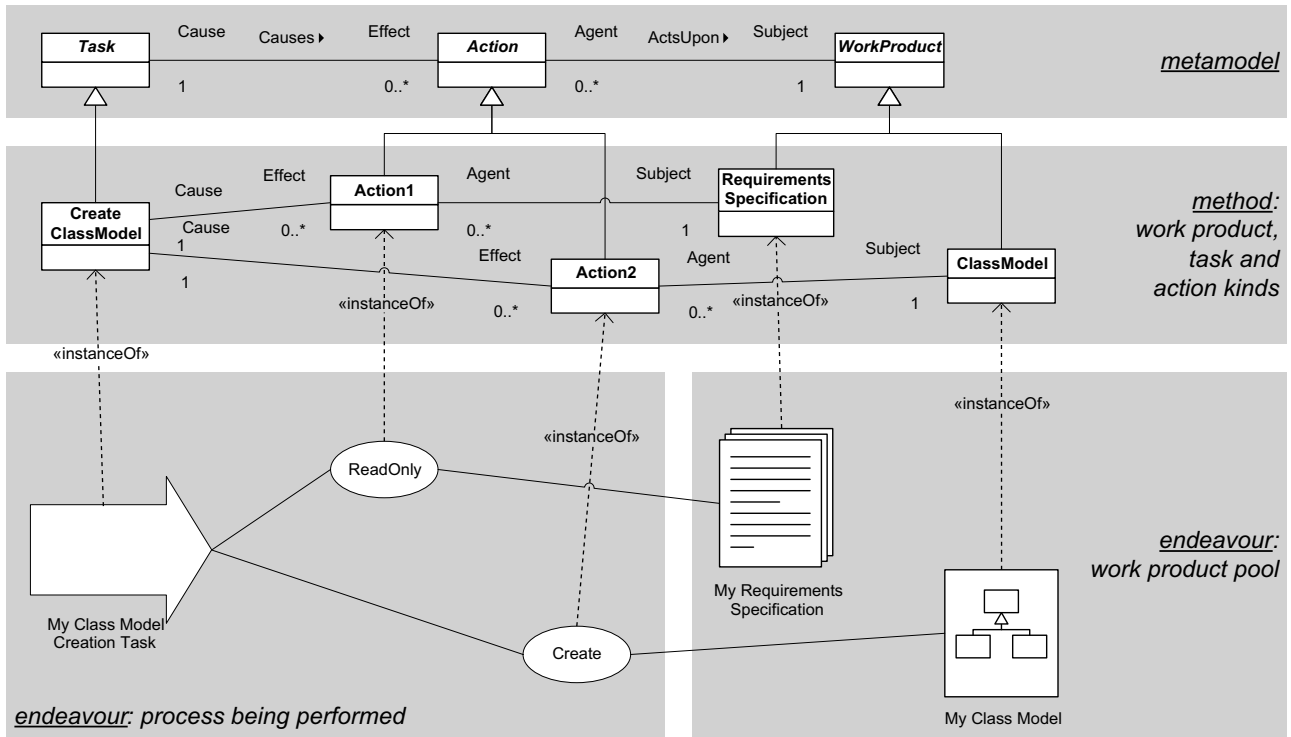


Fig. 6. Modelling actions and action kinds and their interaction with related concepts. Actions are depicted as ellipses labelled with their action type.

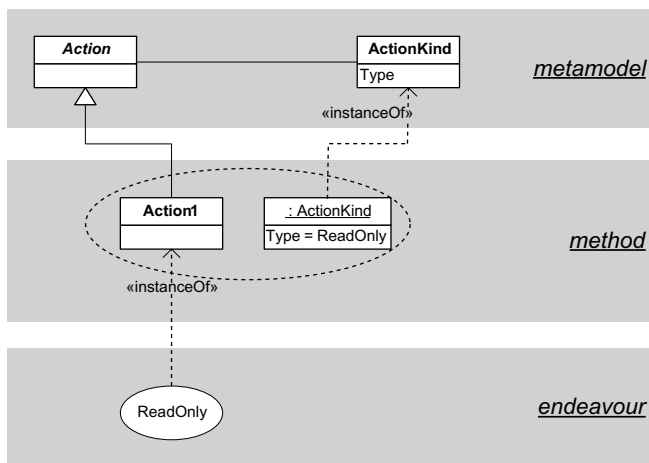


Fig. 7. Action kinds are represented as classes and objects. A “class plus object” entity is called a clabject, and is depicted in this diagram as an ellipse.

action kinds are kinds of actions, so we can say that the *ActionKind* class partitions the *Action* class because instances of the former are equivalent to subclasses of the latter. A class that partitions another class, as in this situation, is called a powertype (Odell, 1994), and the class being partitioned by it, a partitioned type. The pair formed by a powertype and its partitioned type, together with the relationship between them, is called a powertype pattern. These concepts are explored in detail in (Gonzalez-Perez and Henderson-Sellers, 2006a,b,c; Henderson-Sellers and Gonzalez-Perez, 2005a).

3.3. Related work

The approach presented so far in this paper fits nicely with other approaches described by different authors. First of all, the software factories philosophy described by (Greenfield and Short, 2004) includes the concepts of microprocesses, constraint-based scheduling and active guidance as core components. Models (i.e. products) serve as the backbone around which the rest of the software factory is organized, and microprocesses are defined to transform each kind of model. It is easy to see that the process side of the methodology is expressed here as secondary to the more relevant side of products; rather than a central, overarching process, there are multiple, loosely coupled, quasi-autonomous microprocesses that are subject to the needs determined by the relevant products. Furthermore, constraint-based scheduling (Greenfield and Short, 2004) assumes that the actual time ordering of tasks during an endeavour must not be pre-determined, but emerges from the interaction between a complex collection of constraints and the situation of the endeavour at any particular point in time. This is, interestingly, very similar to the network of dependencies established by action kinds plus the work product pool that we have described in previous sections of this paper. Finally, the concept of active guidance (Greenfield and Short, 2004) suggests that guidance to the methodology user must not be given as a pre-determined, rigid network of time-ordered tasks, but must be computed on the fly, again, from the collection of constraints between process and products and the situation

of the endeavour at the particular point in time when the guidance is to be given. The parallel between the work product pool approach and the philosophy of software factories is striking.

The work product pool approach also coincides to a significant extent with Henderson-Sellers' and Edwards' fountain model (Henderson-Sellers, 1992), which proposes that an object-oriented software development environment must be organized as a pool of classes (the products) which are gradually developed by development "jobs" that act upon them. Each development "job" would take one or more classes from the pool, change them as appropriate (perhaps adding some new methods or refactoring some attributes), and put them back to the pool. The pool always contains the latest version of the class collection. The idea of a pool of products plus actions that act upon them is very similar to the ideas presented in this paper.

Historically, the fountain model, although focussing on work products, was perhaps seen as being at odds with the emerging trend towards "process" and, secondly, the idea of a document-driven methodology was equated with the increasingly unfashionable, yet "traditional", plan-based approach to software development. Plan-based approaches were seen as producing documentation "for its own sake" and over the last decade such approaches have been increasingly replaced by more agile approaches focussing on process, people and rapid delivery of the final application (i.e. the software work product). However, despite the increasing adoption of agile methods, some methodology standards, such as (OMG, 2005, 2007), stick to the more traditional approach of clearly discriminating between the (major focus of) process and the data/work product/method content element of a methodology. In modern-day agile processes, the only work product deemed to be of value is this final one; in contrast to those arguing that other work products may be valuable to document the design or to provide the basis for model transformation using a model-driven design approach e.g. (OMG, 2003; Pastor and Molina, 2007).

Another contemporary set of ideas provides other parallels. In contrast to Cartesian philosophy whereby the whole is the sum of the parts, in nature non-linear systems (complex systems) typically exhibit one or more emergent properties, properties of the whole that *cannot* be deduced from the semantics of the parts. In a dynamic system, this means that the behaviour is no longer deterministic, there is no apparent central plan and no co-ordinator role. Since emergent properties of the system cannot be extrapolated from an examination of its constituents, systems theory becomes necessary for such "ecosystems" (whether natural or not – a term also used in (Cockburn and Highsmith, 2001) in the latter case). Introducing chaos theory, especially strange attractors, can provide a tentative link between determinism and stochasticism (Henderson-Sellers, 1996). Viewed holistically, such systems can accommodate extremely large and rich organizational structures. Modern businesses are seen as emergent systems (Lycett

et al., 2003) and, in computer science, the addition of learning techniques, as exemplified by intelligent agents, provides an additional layer of richness. In summary, complex systems cannot be viewed as predictable processes, as demanded by the Tayloristic mindset (Chau et al., 2003; Taylor, 1998), thus shifting the focus away from "process", as does the work product pool approach described in this paper.

At a more abstract level, the work product pool approach also fits nicely with the idea that structure comes first and function comes after. This principle has been used as a key foundation of the Structure-Process-Outcome model (Hohmann, 1997) in the field of psychology of software engineering, claiming that "structure must always come first" and that "without structure there can be no process" (Hohmann, 1997). These ideas have been further explored in the specific context of software development methodologies in (Gonzalez-Perez and Henderson-Sellers, 2006b). A variation on the same theme can be found in the classical tenet of object-oriented programming that one should design their systems around structure rather than function, since structure tends to stay more stable while function tends to change more often (Meyer, 1997). This principle has been used to define the concept of an object as a package of data on which closely-related functions (methods) can operate. This is the (now traditional) object-oriented way of conceptualizing the integration of behaviour and data, i.e. the message format is "object.action" rather than the previous paradigmatic "action(object)". The work product pool approach, similarly, structures the specification of a methodology about those things that change less often, the products, and leaves process as secondary so changing it is easier and not as dramatic. The metaphor is, again,

work_product.process_to_derive_work_product

rather than

process⇒work_product_as_output

Some informal corroboration exists in the observation that class diagrams (structural) are much more frequently used in industry than statechart or sequence diagrams (functional) (Störrle and Knapp, 2007).

One last word must be said in relation to the underpinning assumptions of the work product pool approach. Most current approaches to methodology specification rely on a highly Tayloristic worldview in which humans (and groups of humans) are supposed to act as machines that execute processes (Taylor, 1998). According to this assumption, humans are replaceable parts with little or no individual differences, and the specified process is expected to be executed to the same degree of performance and accuracy by any human that happens to be allocated to it. Conventional management practices (including those that are common nowadays in the software industry) tend

to assume this worldview as true, since, according to (Miller, 2004), it is the only way that managers can obtain the sensation of being in control. Alternative management approaches, such as those proposed by (Thomsett, 2002), deviate from this norm and enter the sometimes called “agile management” field. The work product pool approach, relegating process to a secondary role and laying the grounds for on-the-fly, opportunistic adaptation to work environment changes, embraces a non-Tayloristic management style where people are seen as idiosyncratic individuals that are intelligent enough to decide what they want to do today.

4. Algorithms and techniques

This section describes the procedures to use in order to:

- Determine the candidate tasks that a given developer can perform at a specific point in time, depending on the content of the work product pool and the organizational context.
- Enact the appropriate methodology elements as the developer does her job.

The conceptual infrastructure to be used, as proposed in previous sections, is that of the ISO/IEC 24744 standard metamodel. This metamodel covers many aspects, many of which are not relevant to this discussion, and therefore only a subset of it must be considered here. The relevant subset of ISO/IEC 24744 is shown in Fig. 8.

A few minor changes have been made to Fig. 8 for the sake of readability: irrelevant classes, attributes and associations have been omitted, and role names with maximum cardinalities greater than one have been pluralized (they are expressed in the singular in the standard).

Taking the model in Fig. 8 as our infrastructure, and using pseudo-code as our language, the work product pool of any given endeavour can be defined as:

```
thePool: col WorkProduct
```

This means that the work product pool, denoted by variable *thePool*, is defined as a collection of instances of the *WorkProduct* class. Also using pseudo-code, attributes can be referenced and associations can be navigated. For example, consider the following expressions:

```
ta: new Task
ta.StartTime=Now()
foreach ac in ta.Effects
  ac.Subject.Status=Complete
end
ack: new ActionKind
foreach co in ack.Constraints
  //do something with co.
end
```

The first line creates a new instance of *Task* and assigns it to the variable *ta*. The second line assigns the current date and time (returned by the *Now* function) to the *Start-*

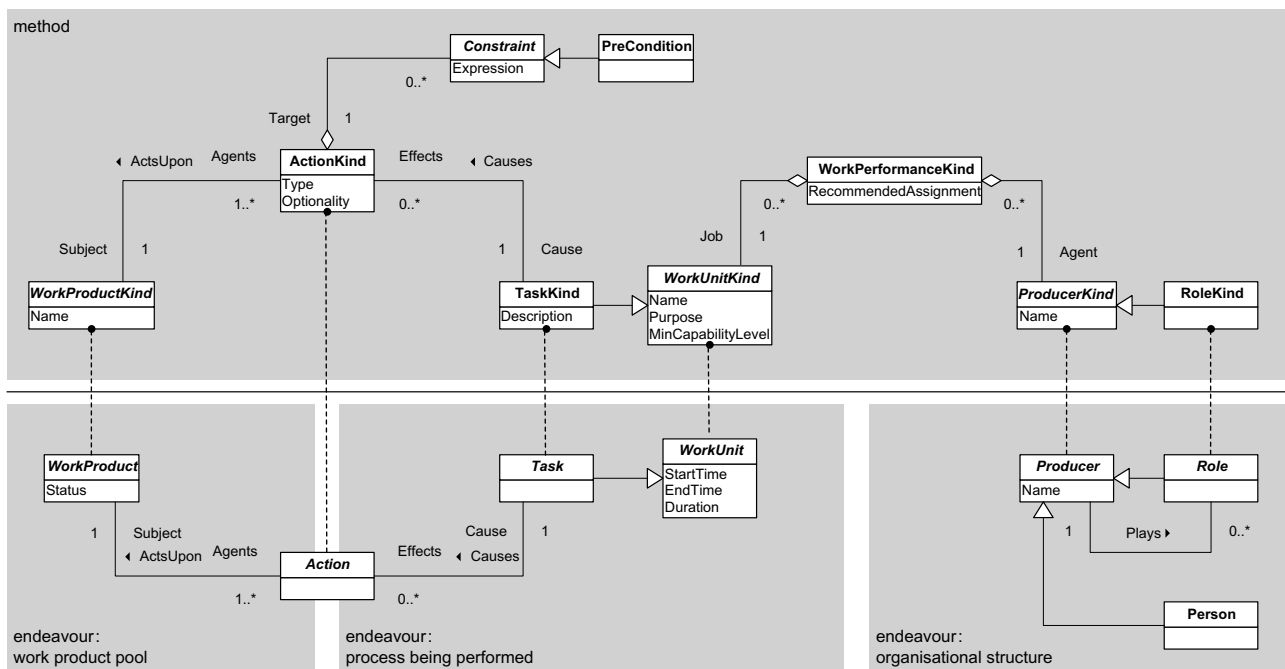


Fig. 8. Subset of ISO/IEC 24744 relevant to the discussion in this section. The classes are laid out according to the domain to which their instances belong: methodology, work product pool, process being performed and organizational structure. Powertype pattern relationships are shown as dashed lines with a round end on the powertype side.

Time attribute of *ta*; in other words, the task represented by *ta* is set to start now. The following three lines show a loop that iterates over all the items in the *Effects* collection of *ta*, which, according to the metamodel, must be zero or more actions and, for each one, navigates to its *Subject* object (of type *WorkProduct* according to the metamodel) and sets its *Status* attribute to *Complete*. In other words, this loop assigns a complete status to all the work products acted upon by the task represented by *ta*.

The last four lines show how a new instance of *ActionKind* is created, and then a loop that iterates over all its constraints. The identifier *Constraints*, used for this purpose, does not exist in the metamodel explicitly; it is obtained by pluralizing the name of the class *Constraint*, associated to *ActionKind*. This illustrates the cases where the association to be navigated is not labelled with a name and the target class does not have a role name; in these cases, the target class name is used, being pluralized if the target maximum cardinality is greater than one.

Finally, we must recall from Section 3.2 that powertype pattern relationships are not conventional associations. Since an instance of a partitioned type is of one single type only, there is an implicit cardinality of one on the powertype side (the one marked with a round end in Fig. 8). Powertype pattern relationships can be navigated as shown by the following pseudo-code:

```
ac: Action
//assign some action to ac.
if ac.Kind.Type == Create
    //do something for creation actions only.
end
```

The previous lines declare a variable of type *Action*, called *ac*, and then assign some existing action to it. The *if* statement in the third line uses the *Kind* identifier to retrieve the instance of the powertype associated to this action, i.e. the corresponding action kind. Once the action kind is obtained, its *Type* can be evaluated and decided upon.

The following sections use pseudo-code and the subset of ISO/IEC 24744 depicted in Fig. 8 to define the algorithms and techniques utilized to determine the candidate tasks that a given developer can perform at a specific point in time, and to enact the appropriate methodology elements as the developer does her job. The φ symbol is used to indicate that a pseudo-code line continues in the next physical line when necessary.

4.1. Determining candidate tasks

Providing guidance to its users is, arguably, the most important objective of any methodology. As we have described in Section 2, traditional approaches to methodology specification conceive giving guidance as providing a process-oriented, time-ordered set of tasks that users of the methodology (i.e. software developers) must follow with as little deviance as possible. In this context, the ques-

tion of “what should I do today?” posed by a developer is usually answered by determining “where we are” in the time-ordered network of tasks and finding the next tasks to be performed. The work product pool approach, on the contrary, suggests that the next task to be performed must be determined not from a pre-defined, fixed plan, but from the contents of the work product pool plus the organizational context. Specifically, the data to be taken into account are:

- The collection of work products in the work product pool, and the properties of each individual work product in this collection.
- The identity of the developer asking “what should I do today?”, especially regarding the roles that she is playing in the endeavour.
- The maturity or capability level of the team in charge of the endeavour, as given by any appropriate scale such as ISO/IEC 15504 (ISO/IEC, 2004) or CMMI (Software Engineering Institute, 2002).
- The methodology being used.

Let us represent the collection of work products in the pool as:

```
thePool: col WorkProduct
```

Similarly, let us represent the developer (or user) and the desired capability level as:

```
theUser: Person
theCapLevel: integer
```

Finally, let us represent the methodology being used as:

```
theMethod: col TaskKind
```

The current user is modelled as an instance of *Person* and the capability level as an integer. The semantics of any particular value of this integer are not relevant to the algorithms described here: the only assumption that is made is that the same semantics (i.e. the same capability or maturity measuring scale) are used throughout the methodology being used and the endeavour being performed.

Given the previous definitions, the question “what should I do today?” can be answered by invoking the following function:

```
function    ComputeCandidateTaskKinds():
col  $\varphi$  TaskKind
```

This function returns a collection of task kinds that is a subset of all the task kinds defined by the methodology being used. They are called “candidate” task kinds because it is guaranteed that any of them can be performed by the current user on the work product pool at the moment of

computation, and within the organizational context given by the relevant capability level, the roles that said user plays in the endeavour, and the chosen methodology. Having said this, we need to clarify that two different types of candidate task kinds can be distinguished: startable and completable task kinds. A startable task kind is one for which at least one action kind is guaranteed to be executable. A completable task kind is one for which all action kinds are guaranteed to be executable. It is easy to see that all completable task kinds are also startable. This distinction is useful because, very often, developers need to start working on a task even if they know that they do not have all the necessary work products and necessary conditions to finish it *at that precise moment*. On the contrary, it is expected that the necessary work products and conditions will be obtained in the near future so that the task will be completed in a reasonable time frame. This fits well with our implicit premise that tasks are not atomic work units; actions are.

The algorithm used to determine which task kinds are candidate, either startable or completable, and which are not, is given by the function body:

```
function ComputeCandidateTaskKinds(): col
TaskKind
  candTaskKinds: new col TaskKind
  foreach TaskKind tk in theMethod
    if not CheckMinCapabilityLevel(tk)
      continue
    end
    if not CheckProducerAllocation(tk)
      continue
    end
    if not CheckActionKinds(tk)
      continue
    end
    add tk to candTaskKinds
  end
  return candTaskKinds
end
```

The function first creates a new, empty collection of task kinds, which will be later filled with the task kinds that are found to be candidates. In order to select the appropriate task kinds, the function iterates over all the task kinds in the methodology and, for each one, three checks are performed: firstly, it is checked that the task kind being considered could be enacted according to its stated minimum capability level. Secondly, it is verified that the current user plays a role that allows her to enact the said task kind. Finally, it is checked that the action kind collection associated to the task kind is satisfiable; in other words, that, at least, one action kind is executable (which would be sufficient to make the task kind startable) by verifying that the necessary work product is present in the work product pool and that the necessary preconditions are met. Once the loops finish, the collection of candidates is returned.

The three checks mentioned in the previous paragraph are implemented as separate functions, which are now described. The first concerns the capability level of the organization and is defined as follows:

```
function CheckMinCapabilityLevel(wuk: WorkUnitKind): boolean
  return
  wuk.MinCapabilityLevel <= theCapLevel
end
```

The function receives as input a work unit kind (such as a task kind) and returns a Boolean value indicating whether or not the work unit kind can be performed according to the organization's capability level. Since, according to the metamodel, each work unit kind states the minimum capability level at which it can be performed, the function simply makes sure that the work unit kind's capability level is not greater than that of the organization. This avoids, for example, that an organization performing at a capability level of 3 tries to carry out work units designed for a capability level of 5.

The second check is related to the organizational structure of the development team, and verifies that the current user is organizationally allowed to perform a given work unit, as defined by the following pseudo-code:

```
function CheckProducerAllocation(wuk:
WorkUnitKind): boolean
  foreach WorkPerformanceKind wek in
wuk.WorkPerformanceKinds
    foreach Role ro in theUser.Roles
      if ro.Kind == wek.Agent
        return true
      end
    end
  end
  return false
end
```

According to the metamodel, producer kinds (e.g. role kinds) can be allocated to work unit kinds via the *WorkPerformanceKind* class. In turn, producers of any type can be assigned to roles. Using this network of objects, the algorithm intersects the collection of role kinds (indirectly) associated to the current user and the collection of role kinds allocated to the potential candidate work unit kind. If the intersection of these two sets is not empty, then the current user is found to be organizationally allowed to perform the work unit kind. The function uses a "lazy" approach, exiting as soon as an intersection is found. If no common role kind is found (i.e. the outermost loop finishes without exiting before), *false* is returned, meaning that the current user is not allowed to perform the work unit kind.

The third check is concerned with making sure that the action kind collection of the task kind being evaluated

can be satisfied. The following pseudo-code implements this:

```

function CheckActionKinds(tk: TaskKind):
boolean
  foreach ActionKind ack in tk.Effects
    if CheckActionKind(ack)
      return true
    end
  end
  return false
end
function CheckActionKind(ack: Action-
Kind): boolean
  if (ack.Type == ReadOnly or ack.Type ==
Modify ↵
or ack.Type == Delete) and ack.Optional-
ity == Mandatory
    if not CheckPoolForAction(ack)
      return false
    end
  end
  if not CheckPreConditions(ack)
    return false
  end
  return true
end
function CheckPoolForAction(ack: Action-
Kind): boolean
  foreach WorkProduct wp in thePool
    if wp.Kind == ack.Subject
      return true
    end
  end
  return false
end
function CheckPreConditions(ack: Action-
Kind): boolean
  foreach pc in ack.PreConditions
    if not Evaluate(pc.Expression)
      return false
    end
  end
  return true
end

```

The *CheckActionKinds* function receives a *TaskKind* object and returns *true* only if the action kinds associated to the given task kind can be satisfied, i.e. if, at least, one action kind is executable. As we said before, if one (but not all) the action kinds are executable, the task kind is called a startable action kind, since a task of that kind can be started (but not completed at this time); if, on the other hand, all the action kinds are executable, the task kind is said to be completable. The *CheckActionKinds* function iterates over all the action kinds of the task kind and, for only those that compulsorily read, modify or delete a

work product (i.e. those that involve some kind of “input”), it calls the *CheckPoolForAction* helper function. This function takes an action kind as input and returns *true* only if the work product pool contains at least one work product of the kind required by any action of the associated kind. After making sure that the required work product exists in the pool, the necessary preconditions are checked by calling the helper function *CheckPreConditions*. Notice that as soon as an action kind exists that is found to be executable, *true* is returned from *CheckActionKinds*, indicating that the task kind being evaluated can be enacted. Whether the task kind is just startable or fully completable can be determined by checking every single action kind associated to it by repeatedly invoking *CheckActionKind*.

If all the checks implemented by the functions described above pass, the task kind being evaluated is added to the collection of candidate task kinds. Thus, and as we explained at the beginning of this section, the top-level function *ComputeCandidateTaskKinds* returns the collection of candidate task kinds for a particular user and a particular organizational context upon completion. Some of the candidate tasks may be completable, while some others may be only startable. A tool implementing this algorithm may go beyond this and mark each individual task kind indicating its type (i.e. only startable or completable), as illustrated in Section 5.

4.2. Enacting methodology elements

Providing guidance to the users of a methodology with regard to what task kinds can be enacted at any point in time is important; however, this guidance is of little use if the task kinds thus identified cannot be easily enacted. Enacting task kinds, obviously, implies enacting other methodology elements and, subsequently, performing a full enactment of the methodology on the relevant endeavour. The algorithms presented in the previous section let us determine, in a fully automatic way, which task kinds are candidate to be enacted at any point in time. It is the user who must decide which particular one she wants to enact at any point in time.

In Section 3 we explained that a methodology is composed of elements, and that each element is a clabject, i.e. a dual entity with a class facet plus an object facet. Taking this into account, enacting a methodology element is, simply, creating an instance of its class facet. For example, let us suppose that the task kind “Document requirements” has been identified as a candidate. This task kind is modelled as a clabject in the methodology, involving a class facet (with a name, for example, *DocumentRequirements-Task*) plus an object facet. If the user decides to enact it (i.e. to start documenting requirements), this means that an instance of *DocumentRequirementsTask* is created. Since, according to the metamodel, a task kind has a collection of associated action kinds, enacting the “Document requirements” task kind entails the automatic enactment

of all the associated action kinds, i.e. the instantiation of the class facets of each associated action kind clabject.

Work product kinds are also enacted, the semantics being similar. Work product kinds are also methodology elements, and therefore are modelled as clabjects. Enacting a work product kind means creating an instance of the class facet of the appropriate clabject. For example, enacting the “Requirements Specification Document” work product kind means that an instance of the *RequirementsSpecificationDocument* class is created.

We must emphasize that enactment does not happen as an upfront activity, i.e. methodology elements are not instantiated in a single go at the beginning of the endeavour. On the contrary, they are gradually instantiated as required, following the algorithm described in Section 4.1. As we said at the beginning of Section 4, methodology elements are enacted *as the developer does her job*.

In addition, notice that not all kinds of methodology elements can be enacted. According to ISO/IEC 24744, only instances of *template* metamodel classes can be enacted. *Resource* classes, on the contrary, cannot be enacted, being used from the endeavour domain as guides, references and instructions (Gonzalez-Perez and Henderson-Sellers, 2006a).

5. Tool implementation

The previous sections have described the work product pool approach; in other words, how methodologies can be exploited by maintaining a work product pool and determining the candidate task kinds at any time by using an automated procedure. This can be carried out by hand or using conventional tools to support specific areas of the approach. For example, a document management system can be used to partially implement the work product pool. However, a specialized tool that implements the algorithms and enactment methods described in Section 4 would yield the maximum benefit. This tool would need to “reason” about the methodology in use; that is, to be able to manipulate a model of the methodology, enumerate its elements and navigate the relationships between them. Furthermore, it would need to “reason” about the endeavour being managed. As with the methodology, the endeavour should be described as a model of entities and relationships. Finally, the two models managed by the tool (methodology and endeavour) would have to be related to each other, so that enacting method elements into endeavour elements is possible. For example, each time that the *Kind* identifier is employed in the algorithms described in Section 4, a transition from the endeavour domain into the method domain is occurring. Fortunately, the ISO/IEC 24744 metamodel comprehensively represents both domains as well as their inter-relationships, and therefore a 24744-based tool would be able to fulfil the model access and navigation requirements that we have outlined in this section. The tool should also implement the candidate task kind determination

algorithms described so that a user can receive guidance about what tasks are ready to be performed at any point.

The MethodComposer prototype toolset (Gonzalez-Perez, 2005) has been used for various purposes over time in our research. MethodComposer consists of the following tools:

- MethodComposer for Engineers, which allows users to maintain method component repositories, create new method components and modify or delete existing ones. Please see Section 3.1 for a discussion of method components.
- MethodComposer for Methodologists, which allows users to assemble methodologies by selecting individual method components from a repository.
- MethodComposer for Developers, which allows users to enact a methodology using the algorithm described in Section 4.1.

These three tools are organized around the three different communities that participate in the method engineering discipline, namely, the method engineers who maintain the repositories, the methodologists who create and maintain methodologies, and the developers who use the methodologies to create final products.

Since this paper focuses on the methodology and enactment domains, the third tool, MethodComposer for Developers, is the most interesting one as far as this paper is concerned, because it is the one that bridges the gap between the methodology as specified and the endeavour as enacted (Fig. 4). The following paragraphs describe how MethodComposer for Developers implements the theory described in this paper and shows how its users can exploit it to answer the question “what should I do today?”

Whenever a user of MethodComposer for Developers wants to enact a methodology, she points the tool to the methodology to be enacted. The methodology is, basically, a model stored on the computer. This model is, naturally, an instance of a metamodel. In our case, ISO/IEC 24744 has been adopted (see Fig. 8 for the relevant subset of this). Once the methodology has been read, a new endeavour is initialized, containing an empty work product pool. The user is then given the chance to select which work product kinds are to be considered internally available and externally provided (see Section 3), and instances of these are created into the pool as required. The work product pool of the newly created endeavour is thus “seeded” so that actions can start operating on them gradually. The tool can display a list with the contents of the work product pool at any time (Fig. 9).

From this moment on, the user can request a list of candidate task kinds to be displayed at any point in time by clicking a button. This list is dynamically determined on the fly by using the algorithm described in Section 4.1, and taking into account, as we have explained, the user’s identity, the contents of the work product pool at the point in time of the request, and the maturity or capability level

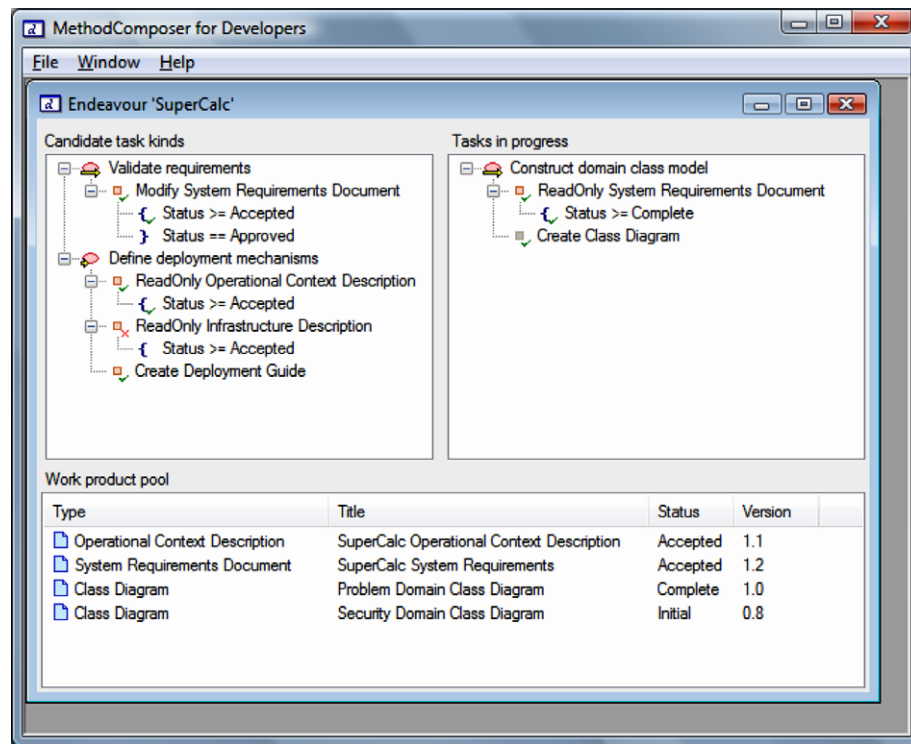


Fig. 9. The endeavour window in MethodComposer for Developers. The work product pool pane appears at the bottom, showing an individual work product per row.

of the team in charge of the endeavour. The list of candidate task kinds is shown to the user in a separate window pane (top left in Fig. 9), which allows the user to examine each task kind, read its purpose, description, minimum capability level and other details via a context menu, and make a decision about which one to enact, if any. This pane also shows, for each candidate task kind, whether it is only startable (shown as a small arrow on the bottom left of the task icon) or fully completable (shown as a long arrow across the whole bottom of the task icon). It also shows, for each task kind, whether each action kind is executable (indicated by a green tick mark) or not (indicated by a red cross). Preconditions are similarly marked depending on whether they are currently satisfied or not.

When the user decides to enact a candidate task kind, the tool creates an instance of the task kind, i.e. a task of the appropriate subtype. For example, if the user decides to enact the “Create class model” candidate task kind, the tool creates an instance of the *CreateClassModel* class, which is a subtype of *Task*. This act of instantiation is what constitutes methodology enactment and, as we can see, occurs gradually and on demand as the user performs her work.

The enactment of action kinds is also supported by the tool. We must recall from Section 3.2 that four types of actions exist: creation, read-only, modification and deletion; this corresponds to tasks that create a new work product, read (but do not change) an existing work product, modify a work product, and delete a work product from the pool. Actions of any type need to be recorded

with the tool, even if they do not alter the work product pool, such as read-only actions; this is because work product availability and preconditions must still be checked and enforced for read-only actions. In order to record actions with the tool, the user can view, at any time, a list of tasks in progress, which appear in a separate window pane (top right in Fig. 9). This list includes all the tasks that this user has enacted and not marked as yet being complete.

From the tasks in the progress window, the user can see, for any particular task, which actions the task is supposed to perform on which work products, and she can notify the system that any particular action has occurred in the real world. For example, if the user is performing a requirements validation task and is about to create a requirements defect report, or has just created it, she can report to the system that an action (“Validate requirements”, “Requirements Defect Report”, Create) must be enacted. Upon this, the tool creates an instance of the appropriate class in the model (i.e. the methodology) and stores it as part of the endeavour. Or, as shown in Fig. 9, if the user is performing a domain class model construction task and is about to create a class diagram as part of that task, she may point to the appropriate action in the tool and use the context menu to enact it. For example, in Fig. 9, the “Create Class Diagram” action kind is displayed with a greyed out icon, meaning that it has not been enacted yet, whereas the “ReadOnly System Requirements Document” action kind is displayed using a coloured icon, meaning that it has already been enacted.

Using the same tasks in the progress window, the user can also notify the tool that a task has been completed. The tool will mark the task as finished and will remove the task from the tasks in the progress window.

This section has described MethodComposer for Developers and how it implements the theory presented in this paper. MethodComposer for Developers is a prototype tool valid only for illustrative purposes, and its functionality is far from being of production quality. However, implementing a theory in a tool provides both a validation of the theory (if it can't be implemented then it is likely to be erroneous) and also provides feedback that improves the theory. This dualistic feedback loop between theory and practice (here a tool) improves both iteratively and can be likened to a grounded theory approach (Glaser and Strauss, 1967), providing justification and initial validation of the theory presented here.

6. Summary and discussion

We have examined here the theoretical basis for current software development methodologies that typically utilize a “process first” mindset. Focussing on process elements can too readily lead to the disadvantages noted by critics of plan-based approaches i.e. that documentation is produced “for the sake of the process” such that all is seen as well if the process is followed irrespective of the quality of the documentation produced. In such process-focussed methodologies, whether traditional plan-based or contemporary agile, a typical enactment starts with delineation of the requirements, then identifies what steps are necessary to move to the next step or phase – this is then applied iteratively until the software application is delivered to the end user.

Secondly, in the area of software process improvement it is assumed that (i) improving the process will inevitably improve the quality of the product and (ii) that the efficacy of the process can be measured by its output i.e. a work product.

Since the real rationale for using a software development methodology is to deliver a high quality software application to the end user, it is clear that the prime focus is initially on this final work product. If we then look backwards (in time) and ask what are the prerequisites to its production, we identify both precursor work products and necessary actions to derive the former from the latter i.e. a process element. Working backwards one work product at a time can therefore create a methodology that moves between nodes representing the various intermediate work products along edges that represent the necessary process steps. That being so, there is a large degree of flexibility made available to the development team since the methodology focusses on specifying the intermediate work products leaving the selection of the specific process step up to the software development team as part of their enactment. Work products are then stored (metaphorically) in a “work product pool” (actually in a software repository)

and abstracted from the pool as and when necessary. The process steps themselves may be individualistically selected by team members depending upon their personal skills sets. This gives a flexibility and agility to the methodology to be used since the methodology is no longer either fixed or “imposed from above”. No extraneous work products are produced, nor are process elements mandated *unless* they contribute directly to the development of the final software product. Of course, when choices are to be made for the “best” method fragment, that fragment must already be in existence (in order that the developer can select it) and some degree of maturity on the part of the team member (augmented if possible by a tool) is required to evaluate options – these are exactly the “challenges” of any method engineering endeavour of course; i.e. this work product pool approach is no more expensive in resources than traditional method engineering.

Using this proposed work product pool approach, we have argued that software development methodologies can be specified that are, at least, as flexible as a lightweight, agile approach and, at the same time, as powerful and scalable as a plan-oriented one. As a test of this theory we have implemented a prototype tool (MethodComposer), which provides a real-life instantiation of the ideas developed here and subjects those ideas to the rigour and formality necessary to implement any idea on a machine (the computer).

Future work will involve an analysis of the impact of this work product pool approach on traditional approaches and standards. We intend to investigate how we might map models based on, say, SPEM or BPMN, to this approach such that optimum synergies can be created.

Acknowledgements

We wish to thank the Australian Research Council for providing funding towards this project. This is contribution number 07/20 of the Centre for Object Technology Applications and Research.

References

- Abrahamsson, P., Salo, O., Ronkainen, J., Warsta, J., 2002. Agile Software Development Methods: Review and Analysis Report, VTT.
- Atkinson, C., 1998. Supporting and applying the UML conceptual framework. “UML” 1998: Beyond the Notation. LNCS 1618. Springer-Verlag, Berlin, Germany, pp. 21–36.
- Atkinson, C., Kühne, T., 2000. Meta-level independent modelling. In: International Workshop on Model Engineering at 14th European Conference on Object-Oriented Programming, 12–16 June 2000.
- Auvinen, J., Back, R., Heidenberg, J., Hirkman, P., Milovanov, L., 2006. Software Process improvement with agile practices in a large telecom company. PROFES 2006. LNCS 4034. Springer-Verlag, pp. 79–93.
- Avison, D., Fitzgerald, G., 2003. Where now for development methodologies? Communications of the ACM 46, 79–82.
- Bettin, J., 2003. Practicalities of implementing component-based development and model-driven architecture. Process Engineering for Object-oriented and Component-based Development OOPSLA 2003 Workshop. COTAR, pp. 19–30.

- Boehm, B., 2002. Get ready for agile methods, with care. *IEEE Computer* 35, 64–69.
- Brinkkemper, S., 1996. Method engineering: engineering of information systems development methods and tools. *Information and Software Technology* 38, 275–280.
- Chau, T., Maurer, F., Melnik, G., 2003. Knowledge sharing: agile methods vs. Tayloristic methods. Twelfth IEEE International Workshop on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE 2003). IEEE Computer Society, pp. 302–307.
- Cockburn, A., Highsmith, J., 2001. Agile software development: the people factor. *IEEE Computer* 34, 131–133.
- Constantine, L., 2002. Non-skid agility? *Information Age*, August/September 2002.
- Dove, R., 1997. The Meaning of Life and the Meaning of Agility (Online resource). <http://www.parshift.com/library.htm>.
- Dowson, M., 1987. Iteration in the software process: review of the third international software process workshop. Ninth International Conference on Software Engineering. ACM/IEEE Computer Society Press, pp. 36–41.
- Glaser, B.G., Strauss, A.L., 1967. *The Discovery of Grounded Theory: Strategies for Qualitative Research*. Aldine Publishing, New York.
- Gonzalez-Perez, C., 2003. OPEN/Metis White Paper (PDF) (accessed 18.02.2003). <http://www.openmetis.com>.
- Gonzalez-Perez, C., 2005. Tools for an extended object modelling environment. 10th IEEE International Conference on Engineering of Complex Computer Systems (ICECCS), 16–20 June 2005. IEEE Computer Society, pp. 20–23.
- Gonzalez-Perez, C., Henderson-Sellers, B., 2006a. On the ease of extending a powertype-based methodology metamodel. Second Workshop on Meta-Modelling (WoMM 2006). LNI P-96. Köllen Druck+Verlag, pp. 11–25.
- Gonzalez-Perez, C., Henderson-Sellers, B., 2006b. An ontology for software development methodologies and endeavours. In: Ruiz González, F., Calero, C., Piattini, M. (Eds.), *Ontologies for Software Engineering and Technology*. Springer-Verlag.
- Gonzalez-Perez, C., Henderson-Sellers, B., 2006c. A powertype-based metamodeling framework. *Software and Systems Modelling* 5, 72–90.
- Graham, I., Henderson-Sellers, B., Younessi, H., 1997. *The OPEN Process Specification*. Addison-Wesley Longman, Harlow, Essex UK.
- Greenfield, J., Short, K., 2004. *Software Factories*. John Wiley & Sons.
- Henderson-Sellers, B., 1992. *A Book of Object-Oriented Knowledge*. Prentice-Hall.
- Henderson-Sellers, B., 1996. Towards a conceptual model of randomness. *Ecological Modelling* 85, 303–308.
- Henderson-Sellers, B., 2003. Method engineering for OO system development. *Communications of the ACM* 46, 73–78.
- Henderson-Sellers, B., 2006. Method engineering: theory and practice. In: Karagiannis, D., Mayr, H.C. (Eds.), *Information Systems Technology and its Applications*. 5th International Conferences ISTA 2006. May 30–31, 2006. Klagenfurt, Austria. Lecture Notes in Informatics (LNI) – Proceedings, vol. P-84, Gesellschaft für Informatik, Bonn, pp. 13–23.
- Henderson-Sellers, B., Gonzalez-Perez, C., 2005a. Connecting powertypes and stereotypes. *Journal of Object Technology* 4.
- Henderson-Sellers, B., Gonzalez-Perez, C., 2005b. The rationale of powertype-based metamodeling. In: Second Asia-Pacific Conference on Conceptual Modelling, 30 January–4 February 2005. Australian Computer Science Communications vol. 7(6). Australian Computer Society, pp. 7–16.
- Henderson-Sellers, B., Nguyen, V.P., 2004. Un outil d'aide à l'ingénierie de méthodes reposant sur l'approche OPEN. *Génie Logiciel* 70, 17–28.
- Henderson-Sellers, B., Serour, M., 2005. Creating a dual agility method: the value of method engineering. *Journal of Database Management* 16, 1–24.
- Henderson-Sellers, B., Serour, M., Gonzalez-Perez, C., Qumer, A., 2007. Improving agile software development by the application of method engineering practices. IASTED International Conference on Software Engineering. ACTA Press, pp. 55–60.
- Highsmith, J., Cockburn, A., 2001. Agile software development: the business of innovation. *IEEE Computer* 34, 120–127.
- Hohmann, L., 1997. *Journey of the Software Professional*. Prentice-Hall, Upper Saddle River, NJ.
- Humphrey, W.S., 1989. *Managing the Software Process*. Addison-Wesley, Reading, MA.
- Humphrey, W.S., Kellner, M.I., 1989. *Software Process Modeling: Principles of Entity Process Models*, Software Engineering Institute.
- ISO/IEC, 2004. ISO/IEC 15504-1: 2004. Software Process Assessment – Part 1: Concepts and Vocabulary.
- ISO/IEC, 2007. ISO/IEC 24744. Software Engineering – Metamodel for Development Methodologies.
- Jacobson, I., Ng, P.W., Spence, I., 2007. Enough of processes – lets do practices. *Journal of Object Technology* 6, 41–66.
- Kruchten, P., 2003. *The Rational Unified Process: An Introduction*. Addison-Wesley, Reading, MA.
- Kumar, K., Welke, R.J., 1992. Methodology engineering: a proposal for situation-specific methodology construction. In: Cotterman, W.W., Senn, J.A. (Eds.), *Challenges and Strategies for Research in Systems Development*. John Wiley & Sons, Chichester, UK, pp. 257–269.
- Lindvall, M., Basili, V., Boehm, B., Costa, P., Dangle, K., Shull, F., Tesoriero, R., Williams, L., Zelkowitz, M., 2002. Empirical findings in agile methods. In: Second XP Universe and First Agile Universe Conference. LNCS 2418. Springer-Verlag, pp. 197–207.
- Lycett, M., Macredie, R.D., Patel, C., Paul, R.J., 2003. Migrating agile methods to standardized development practice. *IEEE Computer* 36, 79–85.
- McBreen, P., 2002. *Questioning extreme programming*. Pearson Education.
- Meyer, B., 1997. *Object-Oriented Software Construction*. Prentice-Hall, Upper Saddle River, NJ.
- Miller, R., 2004. *Managing Software for Growth*. Addison-Wesley.
- Nandhakumar, J., Avison, D., 1999. The fiction of methodological development: a field study of information systems development. *Information Technology & People*, 12.
- Newkirk, J., 2002. Introduction to agile processes and extreme programming. In: *International Conference on Software Engineering 2002 (ICSE 2002)*. ACM Press, pp. 695–696.
- Nguyen, V.P., 2002. A CASE tool for OPEN process construction. Faculty of Engineering, University of Technology, Sydney, Sydney, 597 pp.
- Odell, J., 1994. Power types. *Journal of Object-Oriented Programming* 7, 8–12.
- OMG, 2003. MDA Guide, Object Management Group.
- OMG, 2005. formal/05-01-06. Software Process Engineering Metamodel Specification, version 1.1.
- OMG, 2006. formal/05-07-05. Unified Modelling Language Specification: Infrastructure, version 2.
- OMG, 2007. ptc/07-03-03. Software Process Engineering Metamodel Specification, version 2.0.
- Pastor, O., Molina, J.C., 2007. *OO-Method: Conceptual Model-Based Automated Software Production*. Springer-Verlag.
- Qumer, A., Henderson-Sellers, B., in press. An evaluation of the degree of agility in six agile methods and its applicability for method engineering. *Information and Software Technology*, doi:10.1016/j.infsof.2007.02.002.
- Ralyté, J., Rolland, C., 2001. An assembly process model for method engineering. 13th Conference on Advanced Information Systems Engineering (CAiSE), 6–8 June 2001. LNCS 2068. Springer-Verlag, Berlin, Germany, pp. 267–283.
- Ralyté, J., Deneckère, R., Rolland, C., 2003. Towards a generic method for situational method engineering. 15th International Conference on Advanced Information Systems Engineering (CAiSE 2003). LNCS 2681. Springer-Verlag, Klagenfurt, Austria, pp. 95–110.
- Reifer, D.J., Maurer, F., Erdogmus, H., 2003. Scaling agile methods. *IEEE Software* 20, 12–14.
- Saeki, M., 2003. CAME: the first step to automated software engineering. Proceedings of OOPSLA 2003 Workshop on Process Engineering for

- Object-Oriented and Component-Based Development, 26–30 October 2003. COTAR, Sydney.
- Salo, O., Abrahamsson, P., 2004. Evaluation of agile software development: the controlled case study approach. Fifth International Conference on Product-Focused Software Process Improvement (PROFES 2004). LNCS 3009. Springer-Verlag, pp. 408–423.
- Shere, K.D., Carlson, R.A., 1994. A methodology for design, test, and evaluation of real-time systems. *IEEE Computer* 27, 35–48.
- Software Engineering Institute, 2002. CMMI-SE/SW/IPPD/SS, V1.1, Continuous. CMMI for Systems Engineering/Software Engineering/Integrated Product and Process Development/Supplier Sourcing, Continuous Representation, version 1.1.
- Störrle, H., Knapp, A., 2007. UML 2.0 Tutorial at IASTED 2007 (Online resource). http://www.pst.ifi.lmu.de/~stoerrle/V/UML_2_Folien.rar.
- Taylor, F., 1998. *The Principles of Scientific Management*. Dover Publications.
- Thomsett, R., 2002. *Radical Project Management*. Prentice-Hall, Upper Saddle River, NJ.
- Turk, D., France, R., Rumpe, B., 2005. Assumptions underlying agile software development processes. *Journal of Database Management* 16, 62–87.
- Wong, S.-P., Whitman, L., 1999. Attaining agility at the enterprise level. In: 4th Annual International Conference on Industrial Engineering Theory, Applications and Practice.
- Cesar Gonzalez-Perez** has worked at the European Software Institute and at the Faculty of IT of the University of Technology, Sydney, where he researched in the areas of modelling, metamodeling and methodologies. He is the founder, former technical director and research advisor of Neco, a company based in Spain specializing in software development support services, which include the deployment and use of OPEN/Metis at small and mid-sized organizations. Cesar has also worked for the University of Santiago de Compostela in Spain as a researcher in computing & archaeology, and got his PhD in this topic in 2000. Cesar has published over 30 academic papers in software engineering and other areas.
- Brian Henderson-Sellers** is the Director of the Centre for Object Technology Applications and Research and Professor of Information Systems at the University of Technology, Sydney (UTS). He is author or editor of 27 books and is well-known for his work in object-oriented and agent-oriented methodologies (MOSES, COMMA, OPEN, OOSPICE, FAME), object-oriented metrics and metamodeling. More recently, he has chaired workshops on these topics at OOPSLA and AOIS (Agent-Oriented Information Systems). He is Editor of the *International Journal of Agent-Oriented Software Engineering* and on the editorial board of *Journal of Object Technology*, *Software and Systems Modeling* and *International Journal of Cognitive Informatics and Natural Intelligence*. In July 2001, Professor Henderson-Sellers was awarded a Doctor of Science (DSc) from the University of London for his research contributions in object-oriented methodologies.